Advanced Data Management for Data Analysis Assignment 3

Qinyong Zheng s4325494 Junze Yang s4244532
Yutao Liu s4213211 Zhiwei Ma s4437586
Zhengqi Lang s4264770 Di Xie s4347978

November 10, 2024

1 Environment:

Programming Language: Python
Running system: Windows 11
CPU: Intel® Core™ i7-14700HX
GPU: NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB
Memory (RAM): 32 GB (5200 MHz)

2 Experiment

In this section, five compression techniques are defined: "bin", "rle", "dic", "for", and "dif".
First, the properties of the decoder and encoder instances are initialized using the '-init-' method, including compression type, data type, input file path, and output file path. Then five compression techniques are defined.

2.1 Binary decoding and encoding

Used to converts an integer of the specified type between binary format and numeric format.
Decoding: First, open the data file in binary form, then set up a loop to read the corresponding number of bytes from binfile according to the type specified by data-type, and use struct.unpack to convert the binary data into integers. Finally, write the result to csv.
Encoding: Read each line of the CSV file, traverse each data item in the line, convert each data item (string format) to an integer num, struct.pack converts the integer num to the corresponding binary format and writes it to binfile, and finally outputs the file.

2.2 Run-length encoding and decoding

Run-length encoding reduces the size of data by storing repeated values and the number of repetitions.
Decoding: Open the input file in binary mode, read the value and count pairs in the file continuously, decode the byte data into integer values and counts using ¡q (little endian 64-bit integers), add the decoded (value, count) pairs to the list, call the rle-decode function, decode the encoded data in encoded-data into a complete decoded-data list and write it to the csv file.
Encoding: Define the auxiliary function run-length-encoding to convert the input data in-data into

the run-length encoding format of (value, number of times), open the input CSV file, and read each line in text mode. Use int(line.strip()) to convert each line string into an integer and add it to the data list, pass the read data data to the run-length-encoding function for run-length encoding. Return the encoded-data list, which contains (value, number of times) pairs, indicating each value and the number of times it is repeated continuously, and write it to the binary file.

2.3 Dictionary encoding and decoding

Decoding: The input file is opened in binary mode. The file contents are loaded using the pickle module. The file contains two parts: dictionary and encoded data. The decode-data auxiliary function is then called to decode the encoded data into the original data. The decoded-data is written to a csv file according to the data type specified by data-type.
Encoding: Define the create-dictionary function to generate a dictionary mapping of the data, map each unique value to a unique index, then define the encode-data function to encode the data into an index list based on the generated dictionary, and finally define the dictionary-encoding function to read the content in binary mode and read the data according to data-type and decode it into an integer, call create-dictionary and encode-data to create a dictionary and encode it. Finally, write it to the file.

2.4 Reference frame encoding and decoding

Reference frame encoding reduces storage space by storing the minimum value (reference value) of each data block and the offset relative to the reference value.
Decoding: Read 4 bytes from the input file, representing the reference value, as a benchmark. Then read the offset of a data block, decode the offset data into multiple 32-bit signed integers, get the offset list offsets, add each offset offset to the reference value reference to calculate the original data.
Encoding: First, read the data from the CSV file, convert each value to an integer, and store it in the data list, then open the output file to write the encoded data in binary write mode. Extract a data block of size block-size from data and store it in the block list. Take the minimum value in the current block block and store it in reference as the reference value of the block. For each value in the block, calculate the offset relative to the reference value
(offset = value - reference) and store it in the offsets list, encode reference as a little-endian 32-bit signed integer, and write it to the binary file outfile, finally write the offset and process the next data block.

2.5 Difference encoding and decoding

Decoding: First, the difference is read byte by byte, the number of bytes read is determined according to the specified data-type (int8, int16, int32, int64), and struct.unpack is used to decode it into an integer difference. Then the original value is accumulated: each difference is added to the previous decoded value (previous-value) to get the current decoded original value (current-value), and the previous-value is updated.
Encoding: First try to read the file content in CSV format. If the CSV format reading fails, assume that the file is in binary format and read the content in binary mode. According to the data type specified by data-type, read the corresponding number of bytes one by one and decode them into integers. Then calculate the difference between each data item and the previous data item and write the difference to the output file.

3 Main Program

This is the main interface for data processing, whereby files are encoded and decoded. In this section, we first define the compare-csv method for data alignment, enforce integer format, and remove blank lines. The calculate-compression-ratio function is used to calculate and output the compression ratio and compression rate of the compressed file.
In the main function, the parameters are checked first. The program accepts 4 or 5 parameters. If they do not meet the requirements, they are printed and terminated. The second step is to process the file comparison mode. If the mode is diff, the compare-csv function is called to compare the two CSV files and output whether the file content and structure are consistent. In this mode, the program does not require other encoding or decoding operations. The third step is to process the encoding and decoding modes, obtain the compression type, data type, and input file path, and create a results directory to store the output file. Set the output file path according to the mode. Then record the start time, call the encoding or decoding method to operate, and finally print the result.

4 Observation

In the process of encoding and decoding different files, by setting functions to obtain parameters such as the source file size, compressed file size, compression rate, compression time, decompression time, etc., we can analyze the advantages and disadvantages of different compression methods and their performance through these parameters. The following is an analysis based on different data types:

4.1 DataType: Int8

File	Compression Techinique	Original Size(bytes)	Compressed File(bytes)	Compression Rate(%)	Encoding Time(s)	Decoding Time(s)
l_tax-int8.csv	bin	12002430	6001215	50.00%	1.7274	2.0193
l_tax-int8.csv	rle	12002430	85362800	711.21%	3.2435	3.3037
l_tax-int8.csv	dic	12002430	24032240	200.23%	0.6657	0.6638
l_tax-int8.csv	for	12002430	24192400	201.56%	2.1995	1.5998
l_tax-int8.csv	dif	12002430	6001215	50.00%	2.0142	2.2855
l_quantity-int8.csv	bin	16924405	6001215	35.46%	1.8699	2.2793
l_quantity-int8.csv	rle	16924405	94095952	555.98%	3.1398	3.2681
l_quantity-int8.csv	dic	16924405	33887399	200.23%	0.8900	0.8552
l_quantity-int8.csv	for	16924405	24192400	142.94%	2.2349	1.9248
l_quantity-int8.csv	dif	16924405	6001215	35.46%	1.9150	2.5821
linenumber-int8.csv	bin	12002430	6001215	50.00%	1.7638	2.1398
linenumber-int8.csv	rle	12002430	92592688	771.45%	2.9696	3.1621
linenumber-int8.csv	dic	12002430	24032228	200.23%	0.5213	0.6365
linenumber-int8.csv	for	12002430	24192400	201.56%	1.9296	1.5451
linenumber-int8.csv	dif	12002430	6001215	50.00%	1.7392	2.4150
l_discount-int8.csv	bin	12548425	6001215	47.83%	1.9246	2.3300
l_discount-int8.csv	rle	12548425	87285968	695.60%	2.8411	3.0298
l_discount-int8.csv	dic	12548425	25125121	200.23%	0.5652	0.7200
l_discount-int8.csv	for	12548425	24192400	192.80%	1.9909	1.7311
l_discount-int8.csv	dif	12548425	6001215	47.83%	1.7830	2.2855

1. BIN (Bit Compression):
    - Compression rate is about 35 - 50%, and encoding and decoding time is short.

- Suitable for data with a small range and no obvious repetitive patterns (such as l_tax and linenumber). The effect is poor on l_quantity because the data range is large, resulting in reduced compression efficiency.

2. RLE (Run Length Encoding):

   - The compression rate is the highest, with some files reaching more than 700%, but this means that the compression failed because the compressed file becomes larger.
   - Suitable for data with many repeated values (such as linenumber), but for randomly distributed data (such as l_quantity and l_tax), the metadata overhead is large, resulting in larger files.

3. DIC (Dictionary Encoding):

   - The compression rate is stable at around 200%
   - Suitable for data with high-frequency repeated values(such as l_discount and l_tax). Dictionary encoding can quickly identify and replace high-frequency values, so it performs stably on all files.
     The encoding and decoding time are both short, especially on linenumber and l_discount.

4. FOR (Forward Encoding):

   - The compression rate is about 140%-200%, and the performance is the worst on l_quantity because the data range is large and varies a lot.
   - Suitable for sequences with small changes (such as linenumber). On l_quantity, due to the large difference changes, it cannot be effectively compressed.

5. DIF (Differential Encoding):

   - The compression rate is 35%-50%, and the performance is similar to BIN.
   - Suitable for data with small changes.The encoding and decoding time are average, and the performance is not outstanding.

4.2 DataType: Int16

File	Compression Techinique	Original Size(bytes)	Compressed File(bytes)	Compression Rate(%)	Encoding Time(s)	Decoding Time(s)
l_tax-int16.csv	bin	12002430	12002430	100.00%	1.8839	2.1995
l_tax-int16.csv	rle	12002430	85362800	711.21%	3.3604	3.2540
l_tax-int16.csv	dic	12002430	24032240	200.23%	0.6018	0.6462
l_tax-int16.csv	for	12002430	24192400	201.56%	2.2046	1.6803
l_tax-int16.csv	dif	12002430	12002430	100.00%	2.1931	2.5442
l_suppkey-int16.csv	bin	29341797	12002430	40.91%	2.3153	2.5586
l_suppkey-int16.csv	rle	29341797	96009328	327.20%	3.2311	3.6449
l_suppkey-int16.csv	dic	29341797	58750428	200.23%	1.6782	1.6864
l_suppkey-int16.csv	for	29341797	24192400	82.45%	2.3351	1.8993
l_suppkey-int16.csv	dif	29341797	12002430	40.31%	2.3150	2.7929
l_quantity-int16.csv	bin	16924405	12002430	70.92%	2.2991	2.4802
l_quantity-int16.csv	rle	16924405	94095952	555.98%	3.4201	3.3050
l_quantity-int16.csv	dic	16924405	33887399	200.23%	0.7741	0.8899
l_quantity-int16.csv	for	16924405	24192400	142.94%	2.2285	1.7898
l_quantity-int16.csv	dif	16924405	12002430	70.92%	2.2847	2.6549
linenumber-int16.csv	bin	12002430	12002430	100.00%	2.1102	1.9737
linenumber-int16.csv	rle	12002430	92592688	771.45%	3.0004	2.1538
linenumber-int16.csv	dic	12002430	24032228	200.23%	0.5225	0.6505
linenumber-int16.csv	for	12002430	24192400	201.56%	1.9448	1.6619
linenumber-int16.csv	dif	12002430	12002430	100.00%	1.9739	2.5120
l_discount-int16.csv	bin	12548425	12002430	95.65%	2.3107	2.4042
l_discount-int16.csv	rle	12548425	87285968	695.60%	3.1160	3.0057
l_discount-int16.csv	dic	12548425	25125121	200.23%	0.5855	0.6550
l_discount-int16.csv	for	12548425	24192400	192.80%	2.0900	1.5467
l_discount-int16.csv	dif	12548425	12002430	95.65%	2.1413	2.5575

1. BIN (Bit Compression):
   - For Int16 data, the compression rate of the BIN method is 40%-100%. On I_supplykey and I_quantity, the compression rate is lower (40%-70%)
   - Compared with Int8, because Int16 data occupies more bits, the bit compression effect is poor, especially on data with a large range (such as I_supplykey).
2. RLE (Run Length Encoding):
   - The RLE compression rate is still very high on Int16, and some data even exceeds 700%, indicating that the compression failed. On data with fewer repeated patterns (such as I_supplykey and I_quantity), the metadata overhead is large, resulting in larger files.
   - Compared with Int8, the range and variation of Int16 data are larger, resulting in an increase in the time overhead of RLE.
3. DIC (Dictionary Encoding):
   - Stable at around 200%, similar to the performance of Int8 data. Dictionary encoding can effectively compress high-frequency data, especially on I_linenumber and I_discount.
   - Short encoding and decoding time, good performance. On Int16 data, similar to Int8, dictionary encoding has high search efficiency, so it performs well.
4. FOR (Forward Encoding):

- Average performance on Int16, with a compression rate of about 80%-200%. In particular, the compression rate is low on I_supplykey and I_quantity because these data vary greatly and differential encoding has limited effect.
- Long encoding time and short decoding time. Compared with Int8, the forward encoding time of Int16 is slightly longer because the range and variation of the data are larger.

5. DIF (Differential Encoding):

   - Low compression rate (40%-100%), similar to the BIN method. In the case of large data changes (such as I_supplykey and I_quantity), the effect is not good.
   - The encoding and decoding time is average. Compared with Int8, the time overhead is slightly increased due to the expansion of the data range.

4.3 DataType: Int32

File	Compression Techinique	Original Size(bytes)	Compressed File(bytes)	Compression Rate(%)	Encoding Time(s)	Decoding Time(s)
l_tax-int32.csv	bin	12002430	24004860	200.00%	2.0706	2.2554
l_tax-int32.csv	rle	12002430	85362800	711.21%	3.3436	3.2835
l_tax-int32.csv	dic	12002430	24032240	200.23%	0.6456	0.6410
l_tax-int32.csv	for	12002430	24192400	201.56%	2.2048	1.6233
l_tax-int32.csv	dif	12002430	24004860	200.00%	2.3939	2.6908
l_suppkey-int32.csv	bin	29341797	24004860	81.81%	2.2403	2.6086
l_suppkey-int32.csv	rle	29341797	96009328	327.21%	3.9636	3.6582
l_suppkey-int32.csv	dic	29341797	58750428	200.23%	1.3698	1.7202
l_suppkey-int32.csv	for	29341797	24192400	82.45%	2.5655	2.0696
l_suppkey-int32.csv	dif	29341797	24004860	81.81%	2.5871	3.0387
l_quantity-int32.csv	bin	16924405	24004860	141.84%	2.1581	2.6730
l_quantity-int32.csv	rle	16924405	94095952	555.98%	3.1569	3.3399
l_quantity-int32.csv	dic	16924405	33887399	200.23%	0.7848	0.9255
l_quantity-int32.csv	for	16924405	24192400	142.94%	2.3081	1.6405
l_quantity-int32.csv	dif	16924405	24004860	141.84%	2.4890	2.8049
l_partkey-int32.csv	bin	38675408	24004860	62.07%	2.7009	3.1619
l_partkey-int32.csv	rle	38675408	96019104	248.27%	3.8904	3.8249
l_partkey-int32.csv	dic	38675408	77438883	200.23%	1.8099	2.3851
l_partkey-int32.csv	for	38675408	24192400	62.55%	2.4434	2.3683
l_partkey-int32.csv	dif	38675408	24004860	62.07%	2.6603	3.3819
l_orderkey-int32.csv	bin	46898191	24004860	51.19%	2.5600	3.2598
l_orderkey-int32.csv	rle	46898191	24000000	51.17%	2.0503	2.4522
l_orderkey-int32.csv	dic	46898191	93903154	200.23%	2.2251	2.6501
l_orderkey-int32.csv	for	46898191	24192400	51.58%	2.5750	2.1598
l_orderkey-int32.csv	dif	46898191	24004860	51.19%	2.6100	3.3450
linenumber-int32.csv	bin	12002430	24004860	200.00%	2.1148	2.4764
linenumber-int32.csv	rle	12002430	92592688	771.45%	3.1636	3.1198
linenumber-int32.csv	dic	12002430	24032228	200.23%	0.5268	0.6296
linenumber-int32.csv	for	12002430	24192400	201.56%	2.0450	1.5447
linenumber-int32.csv	dif	12002430	24004860	200.00%	2.2578	2.5321
l_extendedprice-int32.csv	bin	47237224	24004860	50.82%	2.6165	3.3790
l_extendedprice-int32.csv	rle	47237224	96019408	203.27%	3.4802	4.6551
l_extendedprice-int32.csv	dic	47237224	94581997	200.23%	2.4442	2.6451
l_extendedprice-int32.csv	for	47237224	24192400	51.21%	2.4625	2.2218
l_extendedprice-int32.csv	dif	47237224	24004860	50.82%	2.7097	3.3254
l_discount-int32.csv	bin	12548425	24004860	191.30%	2.3029	2.5650
l_discount-int32.csv	rle	12548425	87285968	695.60%	2.9451	3.1053
l_discount-int32.csv	dic	12548425	25125121	200.23%	0.5502	0.6567
l_discount-int32.csv	for	12548425	24192400	192.80%	2.0451	1.5498
l_discount-int32.csv	dif	12548425	24004860	191.30%	2.2269	2.7281

1. BIN (Bit Compression):
   - The BIN method typically has a compression ratio of 50%-81% on Int32 data. For l_extendedprice with a large data range, the compression rate is 50.82%, which is a good effect.
   - Encoding and decoding times are shorter, but slightly faster than Int16. Since Int32 takes up more bits, the time overhead increases.
2. RLE (Run Length Encoding):
   - On Int32, the RLE compression rate is still very high, with some files reaching more than 700%, indicating compression failure.

- Encoding and decoding times are significantly increased. Compared to Int16, when processing Int32 data, the time consumption increases significantly because the data range is expanded, resulting in fewer repeating patterns.

3. DIC (Dictionary Encoding):

   - Still stable on Int32, about 200%. The performance of dictionary encoding is relatively balanced.
   - Encoding and decoding take slightly longer than Int16, but it is still one of the best performing methods. As the data range expands, the dictionary lookup time increases slightly, but the impact is not significant.

4. FOR (Forward Encoding):

   - Compression ratios on Int32 data range from 50%-200%, with 50% indicating good compression, which is a definite improvement over Int16 data, which is typically close to 100% compression.
   - Although the encoding time is slightly longer than Int16 data, this is due to the increased bit width and larger data volume occupied by Int32 data. This increase in time is expected and does not mean any loss of efficiency.

5. DIF (Differential Encoding):

   - On Int32 data, the compression rate of differential encoding (DIF) is 50%-100%, which is relatively stable and even better than that of Int16 data. Int32 data has a wider value range, and when calculating the difference, most changes in the sequence can be captured and expressed as smaller difference values. Therefore, differential encoding is actually more effective on Int32 data.
   - The encoding time increases slightly, but the decoding time remains stable, and it is still a relatively efficient decoding method.

4.4 DataType: Int64

File	Compression Techinique	Original Size(bytes)	Compressed File(bytes)	Compression Rate(%)	Encoding Time(s)	Decoding Time(s)
l_tax-int64.csv	bin	12002430	48009720	400.00%	2.1538	2.3900
l_tax-int64.csv	rle	12002430	85362800	711.21%	3.2610	3.3867
l_tax-int64.csv	dic	12002430	24032240	200.23%	0.6596	0.6643
l_tax-int64.csv	for	12002430	12096072	100.78%	0.2725	1.7530
l_tax-int64.csv	dif	12002430	48009720	400.00%	2.5829	2.9768
l_suppkey-int64.csv	bin	29341797	48009720	163.62%	2.3352	2.7830
l_suppkey-int64.csv	rle	29341797	96009328	327.21%	3.2253	3.6005
l_suppkey-int64.csv	dic	29341797	58750428	200.23%	1.5095	1.5264
l_suppkey-int64.csv	for	29341797	24192400	82.45%	2.4979	2.0135
l_suppkey-int64.csv	dif	29341797	48009720	163.32%	2.7298	3.1603
l_quantity-int64.csv	bin	16924405	48009720	283.67%	2.5521	2.6226
l_quantity-int64.csv	rle	16924405	94095952	555.98%	3.1392	3.3200
l_quantity-int64.csv	dic	16924405	33887399	200.23%	0.8498	0.8797
l_quantity-int64.csv	for	16924405	24192400	142.94%	2.3151	1.7744
l_quantity-int64.csv	dif	16924405	48009720	283.67%	2.3823	3.0874
l_partkey-int64.csv	bin	38675408	48009720	124.14%	2.7648	3.2760
l_partkey-int64.csv	rle	38675408	96019104	248.27%	3.2796	4.0150
l_partkey-int64.csv	dic	38675408	77438883	200.23%	1.7601	2.1597
l_partkey-int64.csv	for	38675408	24192400	62.55%	2.4780	2.3315
l_partkey-int64.csv	dif	38675408	48009720	124.14%	2.9676	3.5204
l_orderkey-int64.csv	bin	46898191	48009720	102.37%	2.7309	3.3656
l_orderkey-int64.csv	rle	46898191	24000000	51.17%	1.9921	2.5348
l_orderkey-int64.csv	dic	46898191	93903154	200.23%	2.1191	2.5323
l_orderkey-int64.csv	for	46898191	24192400	51.58%	2.4301	2.1948
l_orderkey-int64.csv	dif	46898191	48009720	102.37%	2.7377	3.6700
linenumber-int64.csv	bin	12002430	48009720	400.00%	2.3447	3.0564
linenumber-int64.csv	rle	12002430	92592688	771.45%	2.9448	2.7857
linenumber-int64.csv	dic	12002430	24032228	200.23%	0.5648	0.6253
linenumber-int64.csv	for	12002430	24192400	201.56%	1.9776	1.5399
linenumber-int64.csv	dif	12002430	48009720	400.00%	2.2334	3.0564
l_extendedprice-int64.csv	bin	47237224	48009720	101.64%	2.8241	3.4967
l_extendedprice-int64.csv	rle	47237224	96019408	203.27%	3.6298	4.0131
l_extendedprice-int64.csv	dic	47237224	94581997	200.23%	2.3502	2.6650
l_extendedprice-int64.csv	for	47237224	24192400	51.21%	2.5403	2.2149
l_extendedprice-int64.csv	dif	47237224	48009720	101.64%	2.7412	3.8487
l_discount-int64.csv	bin	12548425	48009720	382.60%	2.4703	2.8298
l_discount-int64.csv	rle	12548425	87285968	695.60%	3.0032	3.1023
l_discount-int64.csv	dic	12548425	25125121	200.23%	0.6085	0.6649
l_discount-int64.csv	for	12548425	24192400	192.80%	2.1615	1.5749
l_discount-int64.csv	dif	12548425	48009720	382.60%	2.4241	2.8804

1. BIN (Bit Compression):

   - For Int64 data, the compression rate of bit compression is 100%-400%. For example, l_tax-int64.csv and linenumberint64.csv both reach a compression rate of 400%,which means that compression failed and the file size increased by 4 times.

2. RLE (Run Length Encoding):

   - On Int64, RLE still has a high compression rate, with some files even reaching 700% (such as linenumber-int64.csv and l_tax-int64.csv), indicating that compression has completely failed.

3. DIC (Dictionary Encoding):

    - On Int64 data, dictionary encoding has a compression rate of 200.23%, which is stable and similar to the performance of Int32 data.
    - Encoding and decoding time are short, still one of the most efficient among all methods.

4. FOR (Forward Encoding):

    - On Int64, the compression rate of forward encoding is 50%-200%, and it performs particularly well on data with obvious increasing trends such as l_extendedprice (51.58%).
    - The encoding time is increased because the complexity of calculating the difference sequence increases with the increase of bit width.

5. DIF (Differential Encoding):

    - On Int64, differential encoding has a compression rate of 100%-124%, which is not good, especially on l_partkey-int64.csv, where the compression rate reaches 124%, indicating that the file size has increased.
    - Compared with Int32, the time is significantly increased because the range of variation of Int64 data is wider, and the complexity of calculating differences is also increased.

4.5 DataType: String

File	Compression Techinique	Original Size(bytes)	Compressed File(bytes)	Compression Rate(%)	Encoding Time(s)	Decoding Time(s)
l_shipmode-string.csv	bin	31718249	37719464	118.32%	2.3577	2.8071
l_shipmode-string.csv	rle	31718249	83781057	264.14%	3.7677	4.0376
l_shipmode-string.csv	dic	31718249	63508782	200.23%	1.4396	1.6765
l_shipmode-string.csv	for	31718249	24192400	76.27%	5.8991	1.7400
l_shipmode-string.csv	dif	31718249	-	-	-	-
l_shipinstruct-string.csv	bin	78007624	84008839	107.69%	3.0938	3.7685
l_shipinstruct-string.csv	rle	78007624	107970802	138.41%	3.7050	4.3850
l_shipinstruct-string.csv	dic	78007624	156192830	200.23%	3.9637	4.1088
l_shipinstruct-string.csv	for	78007624	24192400	31.01%	6.3911	2.6208
l_shipinstruct-string.csv	dif	78007624	-	-	-	-
l_shipdate-string.csv	bin	66013365	72014580	109.09%	2.7377	3.4100
l_shipdate-string.csv	rle	66013365	131200322	198.75%	4.2667	6.7400
l_shipdate-string.csv	dic	66013365	132176994	200.23%	2.9076	3.8903
l_shipdate-string.csv	for	66013365	24192400	36.65%	6.3229	2.9203
l_shipdate-string.csv	dif	66013365	-	-	-	-
l_returnflag-string.csv	bin	12002430	18003465	150.00%	2.3943	2.4306
l_returnflag-string.csv	rle	12002430	27294514	227.41%	2.6671	2.4140
l_returnflag-string.csv	dic	12002430	24032204	200.23%	0.7551	0.7897
l_returnflag-string.csv	for	12002430	24192400	201.56%	5.0741	1.6598
l_returnflag-string.csv	dif	12002430	-	-	-	-
l_receiptdate-string.csv	bin	66013365	72014580	109.09%	2.7943	3.8081
l_receiptdate-string.csv	rle	66013365	13162648	198.94%	4.1866	5.7863
l_receiptdate-string.csv	dic	66013365	132176994	200.23%	3.0248	3.5940
l_receiptdate-string.csv	for	66013365	24194200	36.65%	7.3099	2.8426
l_receiptdate-string.csv	dif	66013365	-	-	-	-
l_linestatus-string.csv	bin	12002430	18003645	150.00%	2.1703	2.5168
l_linestatus-string.csv	rle	12002430	10733645	89.43%	1.5748	1.4601
l_linestatus-string.csv	dic	12002430	24032198	200.43%	0.5876	0.6454
l_linestatus-string.csv	for	12002430	24192400	201.56%	4.8797	1.3552
l_linestatus-string.csv	dif	12002430	-	-	-	-
l_commitdate-string.csv	bin	66013365	72014580	109.09%	3.1650	3.6994
l_commitdate-string.csv	rle	66013365	130394022	197.53%	5.7047	6.2203
l_commitdate-string.csv	dic	66013365	132176994	200.23%	3.3580	3.6047
l_commitdate-string.csv	for	66013365	24192400	36.65%	6.1457	2.8649
l_commitdate-string.csv	dif	66013365	-	-	-	-
l_comment-string.csv	bin	164998424	171574037	103.99%	4.9236	6.2169
l_comment-string.csv	rle	164998424	229416800	139.04%	7.5803	8.9576
l_comment-string.csv	dic	164998424	330372442	200.23%	9.5372	8.7946
l_comment-string.csv	for	164998424	24192400	14.66%	15.6534	8.1916
l_comment-string.csv	dif	164998424	-	-	-	-

1. BIN (Bit Compression):

   - On string type data, bit compression performs poorly, and the compression ratio is usually 100%-150%, and there may even be compression failure (such as l_shipmode-string.csv reaching 118.32%).

- String data consists of characters, the ASCII encoding of characters occupies a fixed bit width (8 bits or more), and bit compression cannot remove redundancy when processing characters.

2. RLE (Run Length Encoding):

   - RLE has a higher compression rate on string data with many repeated patterns (for example, l_shipinstruct-string.csv reaches 138.41%), but the effect is poor on random text.
   - Encoding and decoding take the longest time, especially when the time consumption increases significantly on random text data(for example, the encoding time of l_comment-string.csv reaches 9.537 seconds).

3. DIC (Dictionary Encoding):

   - Dictionary encoding performs stably on string data, typically with compression rates of 100%-200%. Works better on short, highly repetitive strings.
   - Encoding and decoding times are short and the most efficient of all methods. Dictionary lookup times are stable even on random text.

4. FOR (Forward Encoding):

   - On string data, forward encoding is better, with compression rates ranging from 14.66%-76.27%. For example, on l_comment-string.csv, the compression rate is only 14.66%, which is significant.
   - The encoding time is the longest because the difference of the character sequence needs to be calculated; the decoding time is shorter.

5. The reason why DIF is not used for encoding&decoding String file:

   - Differential encoding relies on an ordered or increasing sequence of numeric values, whereas string data is typically a sequence of non-numeric characters. The ASCII value changes between characters are random and large, resulting in no significant pattern to be exploited in the differential sequence.
   - In string data, the ASCII value of each character varies greatly. When calculating the difference, additional difference values need to be stored, which increases the file size and worsens the effect.
   - Differential encoding is suitable for numeric and ordered data, but most string type data does not conform to this pattern, resulting in ineffective compression.

5 Conclusion

Through repeated experiments and data comparison, we can draw conclusions and summarize the advantages and disadvantages of different compression methods and the bit width and text types they are suitable for processing.

1. BIN (Bit Compression):

   - is effective for low-range, low-bit-width integer data but shows poor results with high-bit-width and string data.
   - Good for small range integers,but not good for random text and high bit width data.

2. RLE (Run Length Encoding):

   - Performs best with highly repetitive data but struggles with random text and integers.

- It is suitable for highly repeatable integer columns and repeated short sentences, but performs poorly on integer data with large random variations and random text data.

3. DIC (Dictionary Encoding):

   - is the most versatile and stable method, suitable for almost all integer and string data types. The encoding and decoding times are short and the performance is stable.
   - On extremely large ranges or random text, dictionary lookup time will increase slightly, but the impact is not significant.

4. FOR (Forward Encoding):

   - excels with ordered and incremental data, especially date-format strings.
   - The encoding time is longer, especially on high-bitwidth (Int64) data, because the complexity of calculating the difference sequence is high; it is less effective for random character sequences or highly varying integer data.

5. DIF (Differential Encoding):

   - It works well for numerical data with obvious increasing trends (Int8, Int16), and the decoding process is simple and efficient, which is suitable for ordered data.
   - It performs poorly on high-bitwidth (Int64) or character data and cannot effectively capture the changing patterns of strings. Differences in character sequences are usually irregular, resulting in large metadata overhead and compression failures.
   - Not suitable for string data, especially random text or non-numeric strings.

Here is a table of recommended compression methods for different types and bit widths:

Data Type	Bit Width	Best Compression Method	Secondary Method	Not Recommended Methods
High-frequency, Low-range	Int8, Int16	DIC	BIN	RLE
Ordered, Incremental	Int32, Int64	FOR	DIC	BIN, DIF
High-repetition	Int8, Int16	RLE	DIC	BIN, DIF
Large-range, Random Data	Int32, Int64	DIC	FOR	RLE, BIN
Random Text	String	DIC	FOR	BIN, RLE, DIF
Date Format	String	FOR	DIC	BIN, DIF
Repetitive Short Phrases	String	RLE	DIC	BIN, DIF